Meeting – Week 4

Date: 10-17-2026
Meeting Number: 04

Participants

- Neil Saldanha
- Shahala Shahzeen
- Thrupthi Anchan
- Aman Assadi (Team Leader)

Agenda

- Review the Week 4 objective: multi-process simulator and IPC.
- Review the separation into UI, Core and Logging processes.
- Review the IPC mechanism and message protocol.
- Review integration, IPC testing and the benchmark plan.
- Discuss the POSIX IPC requirement and the open items.

Discussion

The team reviewed how the Week 3 simulator was separated into three processes. The UI process uses the existing Swing interface with a RemoteCPU adapter that sends commands to the Core. The Core process keeps the CPU, Memory, Stack and FIFO Queue, and sends log messages to the Logging process.

The team discussed the IPC mechanism and chose Java TCP loopback sockets because the team works on Windows and sockets are available in standard Java. The text protocol was reviewed: PING, LOADSRC, STEP, RUN, RESET and QUIT from the UI, and state snapshots returned by the Core. The use of thread pools in the Core and Logger and the IntegrationLauncher that starts the three processes were also discussed.

The IPC integration test and its result were reviewed. The team noted that TCP sockets are not a POSIX IPC primitive and that this needs to be confirmed against the Week 4 requirement. The benchmark comparison between the standalone and multi-process simulators was discussed, and the values still need to be recorded.

Decisions

- The simulator is separated into UI, Core and Logging processes.
- Java TCP sockets are used for IPC (UI to Core on port 5000, Core to Logger on port 5001).
- The Core and Logger use thread pools to handle connections.
- IntegrationLauncher starts all three processes; IPCIntegrationTest validates the communication.
- The benchmark will use the same assembly program and machine for both versions.
- Confirm the POSIX IPC requirement with the instructor; use a POSIX FIFO version on Linux/WSL if required.

Action Items

Member	Assigned Work
Shahala Shahzeen (Student 1)	UI process and RemoteCPU adapter; check the UI against the Core during the demonstration.
Neil Saldanha (Student 2)	Core process and command server; support benchmark runs on the Core.
Thrupthi Anchan (Student 3)	Logging process and execution.log; check the log output during the demonstration.
Aman Assadi (Team Leader)	Run integration and system tests, record the benchmark values, write the performance analysis, and update the GitHub documentation.

Next Steps

The team will record the standalone and multi-process benchmark values, write the performance analysis, confirm the IPC requirement with the instructor, and update the GitHub documentation before the final demonstration of UI → Core → CPU / Memory / Stack / Queue → Logger.